GRAFOMEM: A Reproducible Benchmark for Agent Memory, and What It Says a Memory Protocol Must Specify

Camilo Ayerbe Posada \ GNS Foundation · ULISSY s.r.l. · grafomem.com

Abstract

Large language model agents increasingly depend on persistent memory, yet there is no shared account of what a memory system must *do*, and so no way to compare or certify one. We present GRAFOMEM, a deterministic, capability-typed benchmark for agent memory. Each of six workloads isolates a distinct capability and is scored against an oracle that derives ground truth — including bi-temporal validity and tenant partitioning — with the safety-critical capabilities, deletion and tenant isolation, measured *two-sided*: for both leakage and over-restriction. The corpus is produced by deterministic generators, gated by independent validators, content-addressed, and regenerated byte-identically across architectures. The benchmark resolves agent-memory behavior into four orthogonal axes — representational capability, embedding quality, retention policy, and a two-boundary privacy primitive — and establishes two cross-cutting results. First, the axes are independent levers: the supersession capability that lifts recall by +0.585 under drift contributes +0.000 under distractor noise, where embedding quality is the only lever, while retention imposes a structural recall cliff no retriever can cross. Second, a capability claim does not certify behavior — backends that advertise hard-delete and multi-tenancy, accept every call, and report success nonetheless leak completely, with no recall footprint to betray them. We conclude that agent memory requires a protocol — specifying first-class versioning, an embedding-agnostic retrieval contract, declared retention, and two-sided read-path privacy — together with a conformance suite, since a declared capability is not a guarantee. The benchmark is the evidence for these requirements and a first draft of the suite.

1. Introduction

Large language model agents increasingly persist state across turns, sessions, and tasks: they remember what a user told them, revise it when it changes, recall it under a token budget, and — at least in principle — forget it on request. This persistent state is now routinely called *memory*, and a growing class of systems exists to provide it — MemGPT/Letta [Packer et al., 2023], Zep

[Rasmussen et al., 2025], mem0 [Chhikara et al., 2025], and framework-native memory modules among them. Yet there is no shared definition of what an agent memory system is supposed to *do*. Each system makes its own implicit choices about whether facts can be superseded, whether history is queryable, what gets evicted under pressure, whether a deletion is honored on read, and whether one user's data can surface for another. These choices are rarely stated, almost never tested against ground truth, and impossible to compare across systems. Memory, the substrate on which agent reliability and user trust both rest, is built ad hoc.

The usual response to "no way to compare" is "build a benchmark," and that is part of what we do. But the deeper problem is not the absence of a leaderboard; it is the absence of *agreement on the axes along which memory systems differ at all*. A recall benchmark that treats memory as a single retrieval task cannot distinguish a system that keeps stale versions of a fact from one that supersedes them, cannot tell whether good recall comes from the retriever or from a generous budget, and — most consequentially — cannot detect that a system which reports a fact as deleted still returns it. The interesting failures of agent memory are not "low recall"; they are *category errors* about what the system is even claiming to provide.

We therefore take a different stance. We construct a benchmark whose workloads are designed to isolate *capabilities* — distinct things a memory system may or may not do — and we measure each one against an oracle-derived ground truth, with backends declared as capability-typed adapters so that what a system *claims* is explicit and separable from what it *does*. Where a capability is safety-critical — deletion and tenant isolation — we measure it *two-sided*: not only whether the system does too little (a forgotten fact resurfaces; a tenant cannot see its own data) but whether it does too much (it purges facts it was not asked to; it withholds legitimate answers in the name of safety). The entire corpus is generated by deterministic procedures, validated by independent checkers, and content-addressed, so that every reported number is reproducible byte-for-byte — which we demonstrate, not merely assert, by regenerating the corpus on a second machine.

Three results structure the paper. First, agent memory behavior decomposes into **four orthogonal axes** — representational capability (can the store version and time-travel over facts?), embedding quality (can it find the right fact among distractors?), retention policy (what does it keep under a budget?), and a **privacy primitive** with two boundaries (deletion and tenant isolation). We show these axes are independent: improving one neither requires nor causes movement in another, and several of the sharpest failures are *embedder-invariant* — they are properties of the store's semantics, untouched by retrieval quality. Second, **a capability claim does not certify behavior.** We exhibit backends that advertise the correct capability flag, accept every relevant call, return success — and still leak: a soft-delete that tombstones a fact yet returns it on retrieval, a multi-tenant store that accepts a tenant scope yet ranks across all

tenants. The type contract is satisfied; the semantic contract is not, and only the benchmark's ground-truth checks tell them apart. Third, and consequently, **agent memory needs a protocol, and that protocol needs a conformance suite.** The benchmark is, in effect, the evidence base for what such a protocol must specify.

Contributions. Concretely, this paper contributes:

1. **GRAFOMEM**, a deterministic, capability-typed benchmark for agent memory spanning six workloads, with an oracle that derives ground truth (including bi-temporal validity and tenant partitioning) and independent validators that gate every published trace.
2. A **reproducibility methodology** for memory benchmarks — deterministic generators, per-workload content-addressed rollup hashes, a locked corpus, and demonstrated **cross-machine byte-identical regeneration** — so that findings are verifiable rather than merely reported.
3. A **four-axis decomposition** of agent memory behavior, with empirical evidence that the axes are orthogonal, including embedder-invariance results that isolate store semantics from retrieval quality.
4. **Two-sided privacy metrics** for the deletion and tenant-isolation boundaries, and the finding that both admit a *claims-but-leaks* failure mode in which an advertised capability is not enforced.
5. A set of **protocol requirements** that follow from the evidence: what a memory standard must specify, and why a conformance suite is a first-class part of it.

Paper structure. Section 2 situates the work against existing memory systems and evaluations. Section 3 specifies the benchmark: the trace model, the oracle, the validators, the metrics, the capability-typed backend interface, and the reproducibility machinery. Section 4 presents the six workloads and their findings along the four axes. Section 5 draws out the two cross-cutting through-lines — that capabilities are orthogonal levers, and that claims do not certify behavior. Section 6 states the protocol requirements the evidence supports. Sections 7 and 8 give limitations and future work, and Section 9 concludes.

2. Related Work

Agent-memory systems. A growing class of systems gives LLM agents persistent memory. MemGPT/Letta [Packer et al., 2023] treats memory as the agent's self-editable state, paging facts between an in-context tier and external storage via tool calls. Zep [Rasmussen et al., 2025], built on the Graphiti engine, stores memory as a *bi-temporal* knowledge graph that tracks both when a fact was true and when it was recorded; mem0 [Chhikara et al., 2025] extracts salient facts into a vector store with an optional graph and a reconciliation step for updates. Earlier work established the broader frame: Generative Agents [Park et al., 2023] introduced a memory stream with reflection, CoALA [Sumers et al., 2023] formalized a taxonomy of agent memory types, and more recent systems

such as A-Mem [Xu et al., 2025] continue to elaborate the design space. Each of these systems makes concrete choices about versioning, retention, and isolation — Zep, notably, commits to a bi-temporal model that directly anticipates our W2 axis — but the choices are embedded in a system and evaluated by downstream task success, not stated as a shared contract or tested capability by capability.

Evaluating memory and long context. The dominant evaluation mode is end-to-end question answering over long conversations. LOCOMO [Maharana et al., 2024] and LongMemEval [Wu et al., 2024] score recall accuracy over extended dialogue histories with an LLM judge; long-context retrieval benchmarks such as "Lost in the Middle" [Liu et al., 2023], the needle-in-a-haystack probes, and RULER [Hsieh et al., 2024] measure whether a model can locate a fact buried in a long input. These treat memory as a single retrieval-or-QA task and report an aggregate score; they do not isolate which capability a system has, do not score against a derived ground truth (the judge is itself a model), and do not test the privacy boundaries at all. That the LOCOMO numbers are openly disputed between vendors — the "benchmark war" over methodology and reported figures — is itself evidence that a single end-task score neither separates what systems can *represent* nor reproduces cleanly.

Foundations the axes import. Each GRAFOMEM axis draws on a mature literature that the agent-memory setting has largely not yet absorbed. The versioning and validity axis rests on bi-temporal data models, where the distinction between valid time and transaction time is decades old [Snodgrass, 1999] (and the SQL:2011 temporal features). The privacy primitive draws on two strands: multi-tenant data isolation from cloud data architecture, and the right-to-erasure and machine-unlearning literature (GDPR Article 17, Regulation EU 2016/679 [European Parliament and Council, 2016]; Bourtoule et al. [2021]), where "delete" is a guarantee about future observability rather than a flag. GRAFOMEM's contribution here is not these ideas but their importation into agent memory as *testable* requirements.

What distinguishes this work. Prior agent-memory evaluations treat memory as a single retrieval-or-QA task, scored end-to-end and, increasingly, by an LLM judge. GRAFOMEM is instead capability-typed — each workload isolates one capability and holds the others fixed — oracle-grounded, with ground truth derived deterministically rather than judged; reproducible byte-for-byte across machines; and, for the safety-critical boundaries, two-sided, scoring both leakage and over-restriction. The consequence is a benchmark that separates what a system can represent from how well it embeds, and that surfaces failures — a store that claims a deletion or a tenant scope and silently ignores it — which no end-task QA score can detect.

3. The GRAFOMEM Benchmark

3.1 Design principles

GRAFOMEM rests on three commitments, each a direct response to a way that ad hoc memory evaluation goes wrong.

Capability-typed. A memory system is not a monolith; it is a bundle of distinct capabilities (can it supersede a fact? query history? honor a deletion? isolate tenants?). We model each backend as an adapter that *declares* a set of capabilities, drawn from a fixed enumeration, and the benchmark keeps the declaration separate from the behavior. This makes it possible to ask the question that matters most — *does a system do what it claims?* — rather than collapsing everything into an aggregate score.

Oracle-grounded. Every trace is generated together with the ground truth it implies. We never label outputs by hand or by judgment; for each query, an oracle derives exactly which facts a correct system must return, from the same event stream the backend will replay. Correctness is thus defined against a deterministic specification, not a reference model's opinion.

Reproducible. The corpus is produced by deterministic generators and content-addressed, so that every number in this paper is reproducible byte-for-byte — a property we demonstrate across machines rather than assert. A benchmark that cannot be independently regenerated cannot be independently checked.

A benchmark instance is then three things: a *corpus* of traces, a set of *backends* (capability-typed adapters), and a *harness* that replays each trace against each backend and scores the results against the oracle.

3.2 The trace model

The atom of the corpus is a **fact**, a quadruple (*predicate*, *subject*, *object*, *valid_from*) carrying additional fields: a monotonic `sequence`, an `importance` weight, an optional `valid_until` and `superseded_by` (for versioning), and an optional `tenant_id`. A fact's identity is content-derived:

$$\mathit{fact\_id} = \text{BLAKE2b-128}(\mathit{predicate} \,\|\, \mathit{subject} \,\|\, \mathit{object} \,\|\, \mathit{valid\_from}).$$

The identity deliberately excludes `tenant_id` and `sequence`. Excluding `sequence` makes re-introducing the same fact idempotent in identity; excluding `tenant_id` means two tenants asserting the same $(S, P, O, \mathit{valid\_from})$ would collide — a property the multi-tenant workload (Section 4) sidesteps by construction, and one we return to as a protocol question in Section 6.

A **turn** is one step of a conversation. It has a role — *user* or *agent query* — free-text `content`, a `timestamp`, and the ground-truth annotations that make

a trace self-describing: a user turn may `introduce` or `delete` facts; an agent-query turn `requires` a set of facts (the facts a correct answer depends on) and may carry an `as_of` timestamp for a historical query. A **session** is an ordered list of turns with a start and end time and an optional `tenant_id`; sessions are the unit of tenancy. All timestamps are microsecond-resolution, timezone-aware ISO-8601, computed rather than sampled from a wall clock so that generation stays deterministic.

3.3 The oracle

The oracle is a pure function from a fact set and session list to ground truth. It replays the annotated event stream in canonical order — ordered by (*timestamp*, *session index*, *turn index*) — and, for each agent-query turn, computes the set of facts that a correct system must be able to return.

Two properties make the oracle more than a bag-of-facts lookup. First, it is **bi-temporal**: a fact is *active* for a query only if it was introduced before the query in *transaction time* (the system has learned it) and holds at the query's *valid time* (it is true in the world at the relevant instant), and has not been deleted. Historical (`as_of`) queries ask about a past valid time, so a superseded fact can be the correct answer to a question about the past while a newer fact is correct about the present. Second, it is **tenant-partitioned**: the active set for a query is computed within the querying session's tenant; facts owned by other tenants are never active.

Formally, let a query q be issued at transaction time t_{tx} within tenant T, with valid-time anchor $t_v = q.as\_of$ if the query is historical and $t_v = t_{\text{tx}}$ otherwise. A fact f is **active** for q, written $f \in \mathcal{A}(q)$, iff

$$intro(f) \le t_{\text{tx}} \wedge f.valid\_from \le t_v < f.valid\_until \wedge \neg\, deleted_{\le t_{\text{tx}}}(f) \wedge f.tenant = T,$$

where $intro(f)$ is the transaction time of f's introducing turn, $deleted_{\le t_{\text{tx}}}(f)$ holds iff a delete turn for f fired at or before t_{tx}, and an absent `valid_until` is taken as $+\infty$. The four conjuncts are exactly the transaction-time, valid-time, deletion, and tenant gates; the ground truth for q is $G(q) = q.requires$.

The oracle is deliberately **strict**: it requires $G(q) \subseteq \mathcal{A}(q)$ and raises otherwise — if a query's `requires` set contains a fact that is not active, because it was deleted, has expired, or belongs to another tenant. This forces a clean separation: a *negative probe* (a query whose correct answer is "nothing," because the tempting fact has been deleted or belongs to another tenant) must declare an empty `requires` set and is scored not as recall but as *leakage* against ground truth (Section 3.6). The oracle emits, per trace, the recall targets for each query (`recall_targets[turn] = requires`) and the deletion times of every deleted fact (`deleted_facts`), which the leakage metrics consume.

3.4 Independent validators

Ground truth derived by the oracle is necessary but not sufficient; a malformed trace can be internally inconsistent in ways the oracle would faithfully but uselessly propagate. A separate validator — independent of both the generator and the oracle — gates every published trace. Among its checks: a fact is never both introduced and deleted in the same turn (V1); a deletion targets a currently-live fact and fires exactly once (V2); every required fact is in principle retrievable at its query — introduced, not deleted, valid, and owned by the querying session's tenant (V4); plus whole-trace consistency and tenant-coherence checks. The validator's universe of "known" facts is the union of surviving and deleted facts, because a trace legitimately carries facts that are later removed (Section 3.7 explains why the deletion workload must retain the full tape). **A corpus build aborts on any violation:** a published trace is, by construction, a validated trace.

3.5 Reproducibility: R1 and R3

R1 — determinism. Every generator is a pure function of (*workload*, *seed*, *difficulty*). All randomness is seeded; identifiers are derived deterministically; timestamps are computed. The same inputs yield the same trace, bit for bit.

Content addressing. Each trace has a content hash, BLAKE2b-256 over its canonical JSON serialization with non-deterministic fields (the random trace identifier and the generation timestamp) removed. The corpus manifest enumerates the published *workload* $\times$ *seed* $\times$ *difficulty* cross-product; the builder expands it, validates each trace, and records into a lockfile: per-trace hashes, **per-workload rollup hashes** (a hash over each workload's sorted trace hashes), and an aggregate **corpus hash** over all sorted trace hashes.

The rollup design has a property we rely on throughout: adding a workload changes the aggregate corpus hash but leaves every *existing* workload's rollup byte-identical. A finding therefore cites its workload's stable rollup, and the suite can grow without invalidating prior citations. We exploit this directly — locking the two privacy workloads into the corpus left the four earlier workloads' rollups unchanged, verified by regeneration.

R3 — cross-machine reproducibility. Determinism on one machine is weaker than it sounds; floating-point summary statistics, library versions, and platform encoding can all leak into "deterministic" output. We therefore regenerate the entire corpus on a second machine of different architecture — the corpus is built and locked on x86-64 Linux and independently regenerated on Apple Silicon macOS — and confirm the per-workload rollups and aggregate hash are byte-identical. R3 is treated as non-negotiable: it is what lets a third party check our numbers rather than trust them.

3.6 Metrics

Retrieval is always scored under a **budget**, a cap on how much the backend may return, approximated by total character count. The character proxy is a deterministic, embedder-independent stand-in for a token budget; sweeping it is often where capability differences surface, since a capable store can win at a tight budget and tie at a saturating one. The core metrics:[1]

metric	definition
M1 recall	fraction of a query's required facts present in the retrieved set, averaged over queries with non-empty targets (negative probes excluded)
M2 precision	fraction of the retrieved set that is required
M3 token cost	retrieved character count (budget proxy)
M7 cross-tenant leakage	fraction of queries whose retrieved set contains a fact owned by a tenant other than the querying tenant

Formally, write $R(q)$ for the set of facts a backend returns for query q under the budget, $G(q) = q.\mathit{requires}$ for the ground-truth targets, and $Q^+ = \{q : G(q) \neq \emptyset\}$ for the positive (non-probe) queries. Then

$$\mathrm{M1} = \frac{1}{|Q^+|} \sum_{q \in Q^+} \frac{|R(q) \cap G(q)|}{|G(q)|}, \qquad \mathrm{M2} = \frac{|R(q) \cap G(q)|}{|R(q)|}.$$

Leakage is defined against a per-query **forbidden set** $\mathit{Forbidden}(q)$ — the facts deleted at or before q (the deletion boundary, W6) or owned by a tenant other than q's (the tenant boundary, W5):

$$\text{leakage} = \Pr_q \left[\, R(q) \cap \mathit{Forbidden}(q) \neq \emptyset \,\right].$$

Both leakage metrics are evaluated on the negative probes whose content most directly tempts the forbidden fact, so a leak is a genuine failure, not an accident of ranking far down a list.

All headline comparisons use a **paired bootstrap confidence interval**. For a claim that a candidate backend beats a baseline, we resample the per-seed

[1] The identifiers follow the metrics module. M4–M6 are reserved for secondary measures (e.g., precision-at-k and budget utilization) used in exploratory analysis but in no headline comparison; we omit them here and retain the module's numbering so the body aligns with the finding cards reproduced in the appendix.

differences (10,000 resamples) and report the point estimate and 95% interval; the claim is sustained only if the lower bound exceeds zero. Reporting intervals rather than point differences guards against over-reading a single seed and makes the strength of each finding explicit.

3.7 Capability-typed backends and the harness

A backend is an adapter implementing a small protocol — `write`, `supersede`, `delete`, `retrieve`, `audit`, `flush` — and declaring a capability set drawn from a fixed enumeration of nine flags (the full set, with the backend-by-capability matrix, is Appendix A); the workloads in this paper exercise five — `SUPERSESSION_CHAIN`, `BI_TEMPORAL`, `HARD_DELETE`, `MULTI_TENANT`, and `AUDIT`. Any operation outside the declared set raises `CapabilityNotSupported`, so a backend's type is an honest statement of what it will attempt.

The harness replays a trace in canonical (*timestamp*, *session*, *turn*) order, **dispatching each operation according to the backend's declared capabilities**. A write becomes a supersession only if the backend claims `SUPERSESSION_CHAIN` and the fact has a recorded predecessor; otherwise it is a plain write, and a drift workload will accumulate stale versions — the failure under test. A delete is a no-op unless the backend claims `HARD_DELETE`. A historical `as_of` query is marked not-applicable, and excluded from that backend's query set, unless it claims `BI_TEMPORAL`. A tenant scope is passed on write and retrieve only to `MULTI_TENANT` backends, so a single-tenant store is never handed a `tenant_id` it would reject. This gating is what lets one trace run against backends of differing capability without special-casing, and it is what guarantees — provably, by the dispatch logic — that introducing the deletion and tenant machinery leaves the capability-free workloads byte-for-byte unchanged (we verify this: the stable-recall floor is identical before and after).

The dispatch only *offers* a capability the chance to act; whether the backend *honors* it is precisely what the oracle-grounded metrics test, and the gap between the two is the subject of Section 5.2.

The reference retriever is a pinned sentence-embedding model (`BAAI/bge-small-en-v1.5`) with exact cosine ranking over stored vectors. For results that must be shown to depend on store *semantics* rather than retrieval *quality*, we re-run with a deterministic bag-of-words **stub** embedder: a finding that is byte-identical under both the reference model and the stub is, by that fact, embedder-invariant — a property of the store, not the retriever. This stub/reference pairing is the instrument behind the orthogonality results of Section 5.1.

3.8 Two structural properties

Two of the findings are not merely empirical observations but consequences of the model, which we state as propositions and confirm in Sections 4 and 5. The first concerns retention; the second is the formal content of "embedder-invariant."

Proposition 1 (bounded-K forgetting cliff). *Consider a store with capacity K under FIFO eviction, on a long-horizon trace in which every fact is a distinct, unambiguously retrievable entity and one fact is introduced per turn. For a query whose target was introduced d facts before it (dependency depth d), recall is*

$$\mathit{recall}(d) = \mathbb{1}[\, d < K \,].$$

That is, recall is exactly 1 *while the target remains within the last K writes and exactly* 0 *once it has been evicted, with the step at $d = K$.* The cliff is a property of the eviction policy and the dependency depth, independent of the retriever; W4 confirms it at $K = 64$ (F9).

Proposition 2 (embedder-invariance). *Let a backend's retrieved set satisfy $R(q) \subseteq \mathit{Visible}(q)$, where $\mathit{Visible}(q)$ is the set of records the store admits as candidates for q, and suppose Visible is determined by the store's bookkeeping (tombstones, tenant tags, capacity) and not by the embedder. Then any predicate over $R(q) \cap \mathit{Forbidden}(q)$ or over $\mathit{Visible}(q) \cap G(q)$ — in particular the leakage and over-restriction outcomes — is identical under any embedding function.* Retrieval quality reorders candidates within $\mathit{Visible}(q)$; it cannot admit a record the store excludes or exclude one it admits. This is why the privacy and retention findings (F9–F13) are byte-identical under the reference model and the stub, and it is the formal sense in which those axes are independent of embedding quality (Section 5.1).

4. Workloads and Findings

The six workloads populate the four axes. Each is generated across five seeds and three difficulty tiers (easy/medium/hard); unless noted, the figures below are at the hard tier, and the privacy results (4.4) are at a 512-character budget over five seeds. Full per-budget sweeps and the verbatim finding cards appear in the appendix; the body reports the headline of each.

4.1 Representational capability (W1, W2)

This axis asks whether the store can *represent* the structure of changing facts: recall a stable fact, supersede an outdated one, and time-travel over history.

W1 — Stable Recall. The simplest workload: facts are introduced and later queried, with no drift, deletion, or distractors. Its purpose is to establish a floor and a frontier. A recency baseline that returns the most recent writes under the budget decays with horizon — recall **0.596** at the medium tier and **0.358** at hard — as older facts fall out of the recency window, while a vector store with the reference retriever recovers them at **1.000**, clearing the deployment threshold on medium and hard and tying on easy, where the floor already saturates (**F1**). Sweeping the budget then traces an efficiency frontier (**F2**): at a one-fact (32-character) budget the reference retriever already ranks the exact target first about 80% of the time (recall **0.801**, precision 0.864), and buying the last fifth of recall up to 1.000 costs roughly 17× the tokens. The efficient operating point

is therefore a *tight* budget — and budget is a first-class axis for everything that follows.

W2 — Drift and Conflict. Here a fact is updated over time ("X lives in A" becomes "X lives in B"), and a *current* query shares its subject and predicate with every stale version. A plain vector store keeps all versions, so at a tight one-fact budget it picks the current one only about 1/depth of the time: recall collapses to **0.281**, against 0.801 for the no-drift W1 baseline. Its apparent recovery to 1.000 at a generous budget is a flood that returns the head *alongside* the stale versions, with no signal distinguishing them — the tight-budget drop is the signature of the missing capability (**F3**). A backend claiming `SUPERSESSION_CHAIN` retires the old version from the candidate set, so a current query sees one candidate per chain and tight-budget recall jumps from 0.281 to **0.867** — a **+0.585** gain at a 32-character budget, attributable entirely to the capability, since the embedder is byte-identical to the baseline's (**F4**). This is the first result in the suite where a *capability, not a better model*, moves the primary metric. Whether *history* stays answerable is a separate question: a `BI_TEMPORAL` backend resolving `as_of` queries against valid-time intervals reclaims the **375** historical queries per seed (hard) that are not low-scoring but *unscorable* for every other backend — turning N/A into ~1.0, a capability that changes which questions are answerable at all (**F5**).

4.2 Embedding quality (W3)

W3 — Distractor Noise. The query's target now sits among semantically similar distractors — facts about the same subject, or the same predicate over different objects — that a retriever must rank against. This is where retrieval quality, not representation, is the lever. Adding distractors imposes a measurable *precision tax* (**F6**): the same store loses precision as distractor density rises. Swapping the embedder is what buys it back — the reference model over a weak baseline recovers **+0.510** recall at a 32-character budget (**F7**) — while capability flags (supersession, bi-temporal, deletion) are *inert* here: they move nothing, because the difficulty is discrimination, not representation. W3 is the clean counterpoint to 4.1 and 4.3: a different axis, a different lever.

4.3 Retention policy (W4)

W4 — Long-Horizon Dependencies. Facts introduced early are required by queries arriving much later, after many intervening writes — so the store must *retain* the right facts under pressure. The trade-off is recall against memory footprint (**F8**): an unbounded store recalls every dependency at 1.000 but its footprint grows linearly with the horizon (here 250 → 4000 facts), while a store bounded at K=64 entries with FIFO eviction holds footprint flat at 64 but sees its *overall* recall erode from 0.659 to 0.348 as a fixed window covers an ever-smaller fraction of a growing history. The sharp result is the **bounded-K cliff** (**F9**): the bounded store recalls correctly up to dependency depth d=63 and drops to zero at exactly $d = K = 64$ — the point where the needed

fact has been evicted. The cliff is *structural*, a function of the eviction policy and the dependency depth, and is **embedder-irrelevant**: a better retriever cannot recover a fact the store has already discarded. Retention is its own axis, upstream of retrieval.

4.4 The privacy primitive (W6, W5)

The fourth axis is safety-critical and, unlike the others, *two-sided*: a store can fail by returning what it must not (leakage, a false positive) or by withholding what it must return (over-restriction, a false negative). It has two boundaries — deletion and tenant isolation — and we find them structurally identical.

W6 — Deletion / Leakage. A fact is introduced, later deleted, and then a query tempts it directly. A correct store returns it before deletion and never after, while leaving same-subject survivors intact. Four backends span the surface:

backend	leakage	survivor recall	verdict
vector_only	1.000	1.000	LEAKS
soft_delete	1.000	1.000	LEAKS
honest_delete	0.000	1.000	PASS (both)
coarse_delete	0.000	0.000	OVER-DELETES

`vector_only` has no deletion at all, so the deleted fact persists and leaks. `soft_delete` *claims* `HARD_DELETE`, tombstones the fact, reports success — and still returns it on retrieval, because the tombstone is never checked on the read path (**F10**): the capability is advertised and unenforced. `honest_delete` removes the fact from store and index and passes both directions. `coarse_delete` deletes by subject, so it leaks nothing but purges same-subject survivors, driving survivor recall to zero — the over-deletion failure (**F11**). Paired bootstrap intervals confirm both directions: leakage, soft − honest, **+1.000** [+1.000, +1.000]; survivor recall, honest − coarse, **+1.000** [+1.000, +1.000]. The result is embedder-invariant: which records survive a delete is decided by the store, not the retriever.

W5 — Tenant Isolation. Many tenants share one store; every subject exists under every tenant with a different object, so a query in tenant A ("Where is Kesia located?") is byte-identical to tenant B's and a tenant-blind store ranks the wrong tenant's answer at the top. The same four-way structure appears:

backend	leakage	in-tenant recall	verdict
vector_only	1.000	1.000	LEAKS
leaky_tenant	1.000	1.000	LEAKS
tenant_scoped	0.000	1.000	PASS (both)
over_isolating	0.000	0.000	OVER-ISOLATES

`leaky_tenant` is the exact analog of `soft_delete`: it claims `MULTI_TENANT`, accepts a tenant scope on every call, tags each fact's owner — and then ignores the scope on retrieval, leaking identically to the no-capability baseline (**F12**). `tenant_scoped` filters retrieval to the querying tenant and passes both directions. `over_isolating` adds an over-cautious heuristic — withhold any subject that appears under more than one tenant — and, because W5 shares every subject, withholds everything, collapsing in-tenant recall to zero (**F13**). Intervals: leakage, leaky − scoped, **+1.000** [+1.000, +1.000]; in-tenant recall, scoped − over, **+1.000** [+1.000, +1.000]. On the reference embedder the leaky backends leak at 1.000 *while holding recall at 1.000* — leakage and recall are independent, so a privacy failure leaves no recall footprint to warn you.

The matched pair. The two boundaries are the same object seen twice:

boundary	leakage (false positive)	over-restriction (false negative)	claims-but-leaks backend
deletion (W6)	forgotten fact resurfaces	survivors purged	`soft_delete` (F10)
tenant (W5)	other tenant's fact returned	own facts withheld	`leaky_tenant` (F12)

Each boundary is two-sided, embedder-invariant, and admits a backend that advertises the correct capability and fails anyway. Together they are the privacy primitive — and they motivate the two cross-cutting arguments of Section 5.

5. Cross-cutting Analysis

The workload-level findings of Section 4 are, individually, statements about one capability at a time. Read together they support two claims that no single workload makes on its own, and that are the paper's reason for being a paper rather than a results table: the axes are *orthogonal*, and a capability *claim* does not certify *behavior*.

5.1 Orthogonal levers, not one knob

The four axes — representational capability, embedding quality, retention policy, and the privacy primitive — are independent in a strong, testable sense: moving one neither requires nor produces movement in another. The instrument that lets us assert this rather than merely assume it is the stub/reference embedder pairing (Section 3.7). A finding that is byte-identical under the reference model and under the bag-of-words stub is, by that fact, a property of store *semantics*, not of retrieval *quality* — and across the suite the structural findings are exactly those that survive the swap.

The sharpest single demonstration is one lever measured on two workloads. On W2, the `SUPERSESSION_CHAIN` capability lifts tight-budget recall by **+0.585**

with the embedder held byte-identical. On W3 — where signal and distractor are structurally identical, so there is nothing for a capability to reshape — the same lever measured the same way is **+0.000**, asserted per seed as exact equality: `supersession_chain` and `bi_temporal` are bit-for-bit `vector_only`. A lever that triples recall on one workload and does *nothing* on another is, by construction, not the same knob as whatever does move W3 — which is the embedder, lifting recall **+0.510** at the same budget while the capability ladder stays flat. Capability and embedding quality are thus shown to be complementary, not substitutable: the one moves with the other pinned, and vice versa.

Retention sits upstream of both. On W4 the embedder is effectively perfect (every entity is unique, so a stored fact is retrieved at 1.000) and the supersession and bi-temporal capabilities are irrelevant, yet the backends diverge enormously — purely by what they retain — and the bounded-K cliff falls at $d = K = 64$ *regardless of embedder*: the stub reproduces the cliff at the identical depth, because no retriever can return a fact the store has already evicted. And the privacy primitive is embedder-invariant on both boundaries: which records survive a deletion or a tenant filter is decided by the store, so `soft_delete` and `leaky_tenant` leak at 1.000 under the reference model exactly as under the stub.

The practical reading, which the per-workload cards converge on independently, is that one must **diagnose the failure mode before choosing the fix**. Staleness or drift is repaired by adding a capability (supersession, valid-time), not by a larger embedder; confusion among plausible look-alikes is repaired by a better embedder, not another capability; unaffordable scale is a retention-policy choice; and a privacy breach is an enforcement question, untouched by any of the three. A benchmark that varied a single axis — as most agent-memory comparisons do — would report at most a quarter of this surface and would routinely prescribe the wrong remedy.

5.2 A capability claim does not certify behavior

The capability-typed design (Section 3.7) deliberately separates what a backend *claims* from what it *does*: the harness dispatches an operation to a backend that declares the corresponding capability, but whether the backend honors the operation is left for the oracle-grounded metrics to decide. Two backends turn that separation from a design nicety into a finding.

`soft_delete` claims `HARD_DELETE`. It accepts every `delete` call, tombstones the target, and returns success; its `audit` view even reports the fact as gone. Yet it leaks at **1.000**, because the read path never consults the tombstone — and so its own audit and its own retrieval *disagree*. `leaky_tenant` claims `MULTI_TENANT`. It accepts a tenant scope on every write and read, tags each fact with its owner — and leaks at **1.000**, because retrieval ignores the scope, behaving identically to the no-capability baseline. In both cases the *type* contract is impeccable: the capability is advertised, every relevant call is accepted, success is returned. The

semantic contract is broken, and nothing in the interface reveals it.

What makes this more than a pair of bugs is that the failure is invisible from inside the system. On the reference embedder, `leaky_tenant` leaks at 1.000 while holding in-tenant recall at 1.000: the store is simultaneously perfectly useful to its owner and perfectly porous to its neighbour, and there is no recall footprint to raise an alarm. `soft_delete` actively certifies its own compliance through `audit`. A system that trusted declared capabilities — or even self-audit — would ship both. Only an external, oracle-grounded, *two-sided* check separates the backend that honors a capability from the one that merely claims it.

The consequence is the paper's bridge to Section 6. A capability flag is necessary but not sufficient, so a memory protocol cannot establish conformance by inspecting declared capabilities; it requires a **conformance suite** — an executable, ground-truth test that each claimed capability is actually enforced, in both the leakage and the over-restriction directions. This is not an optional appendage to a protocol. `soft_delete` and `leaky_tenant` are the existence proof that the suite is load-bearing, and the two-sided privacy workloads of Section 4 are a first draft of exactly such a suite.

6. Implications for a Memory Protocol

The benchmark was built to answer a design question as much as an empirical one: if agent memory needs a protocol, what must that protocol specify? The evidence of Sections 4 and 5 supports a set of *requirements* — the dimensions a memory standard cannot leave unspecified — without yet committing to a particular *specification* of each, which we defer to a follow-up (Section 8). We state five, each tied to its evidence.

Versioning and validity must be first-class (W2; F3–F5). A protocol that treats memory as an append-and-retrieve log cannot express the most common real-world event — a fact changing — and a store built on it returns stale data alongside current data with no signal to separate them (F3). The protocol must define what it means to *supersede* a fact (retire a predecessor from the answer set) and to query the past (resolve a query against valid time): these are the operations that recover tight-budget recall (F4) and make historical questions answerable at all (F5). They are representational commitments, independent of retrieval quality.

Retrieval must be embedding-agnostic (W3; F6–F7). Embedding quality is a genuine lever — it is the only thing that moves discrimination under noise (+0.510) — but it is orthogonal to every other axis and improves on the timescale of model releases, not protocol revisions. A protocol should specify the retrieval *interface* and a *budget contract* (what a query costs, what a store must rank) while deliberately not mandating an embedder. Fixing a model into the standard would couple an axis the evidence shows is independent, and date the standard to a model generation.

Retention policy must be declared, not implicit (W4; F8–F9). Every store makes a retention choice — unbounded, bounded, compacting — and that choice imposes a hard, structural limit on the answerable horizon (the cliff at $d = K$) at a footprint cost the consumer must be able to reason about. A protocol must let a store *declare* its retention policy and the footprint/coverage contract it implies, so that an agent depending on a long-ago fact knows whether the store can still be expected to hold it. Retention is a first-class axis, not an implementation detail.

Privacy must be two-sided and enforced on the read path (W5, W6; F10–F13). The deletion and tenant boundaries are structurally identical and each fails in two directions: leakage (returning what must not be returned) and over-restriction (withholding what must be). A protocol must specify both boundaries' semantics — what a deletion guarantees, what a tenant scope guarantees — and must locate the guarantee on the *read path*, because the failures we observe are exactly stores that accept the write-side call (a tombstone, a tenant tag) and ignore it on retrieval. A flag that a store "supports deletion" or "supports tenancy" is not a guarantee; the guarantee is a property of what retrieval returns.

A conformance suite is part of the protocol (§5.2). Because a capability claim does not certify behavior — `soft_delete` and `leaky_tenant` advertise the correct flag, satisfy the type contract, and leak — a protocol cannot define conformance by inspecting declared capabilities. It must ship an executable, oracle-grounded conformance suite that tests each claimed capability in both directions, and define "supports capability X" to mean "passes the suite for X," not "declares X." The two-sided workloads of this paper are a first draft of that suite.

One requirement the model deliberately leaves open. Fact identity in our trace model is content-derived and excludes the tenant (Section 3.2), so two tenants asserting the same triple would collide absent the distinct-object construction we use. A protocol must decide whether memory identity is content-only or tenant-scoped — a choice with direct consequences for deduplication, isolation, and the deletion semantics above. We surface it as an open specification question rather than resolve it here.

Together these requirements form the evidence-grounded skeleton of a memory protocol: what it must specify, and — through the conformance suite — how a claim of support is to be verified. The candidate *semantics* for each (a concrete supersession model, a deletion-and-tenancy enforcement model) are the subject of the follow-up; the contribution here is to establish, empirically, that these are the axes a standard cannot leave unspecified.

7. Limitations

The findings should be read with several constraints in view, none of which we believe undermines the central claims but each of which bounds their scope.

Synthetic traces. The corpus is procedurally generated, not drawn from real agent transcripts. This is what buys determinism, oracle-derived ground truth, and byte-level reproducibility — properties a corpus of captured conversations cannot offer — but the distributions of facts, drift, distractors, and queries are ours. The results are claims about *capabilities and their failure modes* (a store either honors a deletion or it does not), which we argue are robust to distribution; the absolute numbers are not production estimates.

Character-budget proxy. The retrieval budget is enforced in characters rather than through a real tokenizer (the `cl100k_base` wiring is deferred). Ratios are valid because every backend shares the unit, and the budget-as-axis findings are unaffected, but absolute tokens-per-fact are not spec-faithful.

A single headline embedder, and a lexical stub. Headline numbers use one pinned model (BGE-small-en-v1.5). Two things mitigate the dependence: the embedder- invariance results (the structural findings hold under the bag-of-words stub), and the stub cross-checks themselves. But the embedding-quality axis (W3) is anchored by that lexical stub, which conflates "semantic vs lexical" with the stub's lack of stemming; a second, graded real embedder would turn the quality axis from binary into a curve and sharpen "how much quality buys how much recall."

No LLM in the loop. We score retrieval against ground truth, not end-to-end agent task success. Whether a leaked or dropped fact actually changes an agent's output is downstream of what we measure; the memory layer is tested in isolation, by design, so that a failure is attributable to the store rather than to a generator's idiosyncrasies.

Bounded difficulty and a single eviction policy. The hard tier is finite (four tenants, ~4000-fact horizons, a 7.3× distractor haystack); a larger stress tier is feasible but unrun. The only bounded retention strategy evaluated is FIFO eviction, so the cliff we report is FIFO's cliff; importance- or frequency-weighted policies, and compaction, are untested here.

8. Future Work

The limitations point directly at the next steps, several of which the workload findings explicitly surface.

A compaction backend. The bounded-K cliff (F9) is the cliff of a store that *drops* old facts. A backend that instead *merges* them — summarizing or consolidating rather than evicting — would test whether the cliff can be softened: recall held over distance at sub-linear footprint. It is the natural next backend, and because merging interacts with deletion and supersession, it should be evaluated against W4 and W6 together.

The conformance suite as a standalone artifact. Section 6 argues the suite is part of the protocol; the two-sided privacy workloads are its first draft. The next step is to package the oracle-grounded, two-sided checks as a runnable

conformance harness that any third-party store can be driven through, reporting per-capability pass/fail in both the leakage and over-restriction directions.

A graded embedding axis. Replacing the lexical stub with a second, weaker real sentence model converts W3's embedder comparison from binary (semantic vs lexical) into a quality gradient.

Scale. A 50k-fact stress tier — feasible, since the oracle processes 4000 facts in roughly a tenth of a second — would confirm the linear-footprint slope holds an order of magnitude further.

End-to-end and real-world validation. Two complementary extensions: scoring agent task success on top of the memory layer (closing the no-LLM-in-loop gap), and validating the synthetic findings against captured agent transcripts.

Candidate protocol semantics. The companion to this paper is the specification the requirements of Section 6 imply: a concrete supersession-and-validity model, a deletion-and-tenancy enforcement model, and a resolution of the content-only versus tenant-scoped identity question that the trace model leaves open.

9. Conclusion

Agent memory is the substrate on which agent reliability and user trust both rest, and it is built ad hoc: each system makes its own unstated choices about versioning, forgetting, and isolation, untested against ground truth and impossible to compare. We have argued that the way to discipline this is not another leaderboard but a benchmark designed to expose the *axes* along which memory systems differ — and we built one: deterministic, capability-typed, oracle-grounded, and reproducible byte-for-byte across machines.

The benchmark yields four orthogonal axes — representational capability, embedding quality, retention policy, and a two-boundary privacy primitive — and two cross-cutting results. The axes are independent levers, not one knob: the same `SUPERSESSION_CHAIN` capability that lifts recall +0.585 on drift moves it +0.000 under distractor noise, where the embedder is the only lever, while retention sets a structural cliff that no retriever can cross. And a capability claim does not certify behavior: backends that advertise `HARD_DELETE` and `MULTI_TENANT`, accept every call, and report success nonetheless leak at 1.000, with no recall footprint to betray them.

Together these say what a memory protocol must specify — first-class versioning and validity, an embedding-agnostic retrieval and budget contract, a declared retention policy, and two-sided privacy enforced on the read path — and, decisively, that the protocol must ship a conformance suite, because a declared capability is not a guarantee. The benchmark is at once the evidence for those requirements and the first draft of that suite. The specification they imply is the work that follows.

Appendix A — Backend capability matrix

The interface (`interface.py`, v0.1.1) defines nine capability flags: `BI_TEMPORAL`, `HARD_DELETE`, `SUPERSESSION_CHAIN`, `CROSS_SESSION_PROPAGATION`, `MULTI_TENANT`, `CONFLICT_DETECTION`, `PROVENANCE`, `CRYPTOGRAPHIC_PROVENANCE`, and `AUDIT`. The eleven reference backends and the capabilities each declares (columns omitted are claimed by no backend evaluated here). Capability columns are abbreviated — SC = `SUPERSESSION_CHAIN`, BT = `BI_TEMPORAL`, HD = `HARD_DELETE`, MT = `MULTI_TENANT`, AU = `AUDIT`; · = not declared, ✓ = declared:

backend	SC	BT	HD	MT	AU	tested on
`persistence` (floor)	·	·	·	·	·	W1–W4
`vector_only`	·	·	·	·	✓	W1–W6
`supersession_chain`	✓	·	·	·	✓	W2 (inert on W3)
`bi_temporal`	✓	✓	·	·	✓	W2 (inert on W3)
`bounded_vector` (K=64)	·	·	·	·	✓	W4
`soft_delete`	·	·	✓	·	✓	W6
`honest_delete`	·	·	✓	·	✓	W6
`coarse_delete`	·	·	✓	·	✓	W6
`leaky_tenant`	·	·	·	✓	✓	W5
`tenant_scoped`	·	·	·	✓	✓	W5
`over_isolating`	·	·	·	✓	✓	W5

`bi_temporal` declares `SUPERSESSION_CHAIN` in addition to `BI_TEMPORAL` (it is a strict superset of `supersession_chain`). Note that the three delete backends are capability-*identical* ({HARD_DELETE, AUDIT}), as are the three tenant backends ({MULTI_TENANT, AUDIT}): F10–F13 turn precisely on identical declarations yielding divergent behavior, separable only by the two-sided metrics. The interface anticipates exactly this — it reserves a `ConformanceViolation` exception for the case where declared capabilities do not match observed behavior (design anchor B2). Four flags — `CROSS_SESSION_PROPAGATION`, `CONFLICT_DETECTION`, `PROVENANCE`, and `CRYPTOGRAPHIC_PROVENANCE` (the last backed by an Ed25519-over-`fact_id` verifier in the interface) — are defined but exercised by no reference backend here, and are left to future workloads.

Appendix B — Corpus provenance

Suite `grafomem-bench-v0.1.8`: 90 traces (6 workloads × 5 seeds × 3 difficulties), 55,014 turns, 15,342 queries. Each trace is content-addressed by BLAKE2b-256 over its canonical JSON (excluding the random trace identifier and the generation timestamp); a per-workload rollup hashes that workload's sorted trace hashes, and the corpus hash aggregates all sorted trace hashes.

`corpus_hash: 8ea8edc8d34f93689ae8b3343b7f54fdd832a6ce101c8f6a08cabd2c88baedb1`

workload	rollup hash
W1	`3028af2d728db47c18ef6353d3881302ad527dcbdb6bc7370f9df09394`
W2	`5a39a2ebc18a412330b9316d38b66233ff6155c8befdb2c56ae7b16912`
W3	`2feea9abd2c86de6dd8df87acccd25a4551810bef23cc58ceef4049598`
W4	`cd265abad10e8d0af69427f6a3b3abf062f89777ef1758ec51d06705d4`
W5	`a791dae52e917340708da237205e005c7abaa1f5d52c0661cc20fa864c`
W6	`a758332bb77c4422a8074fdc71db309de06786b71d20699de601a86807`

The rollups are stable under suite growth: locking W5 and W6 left the W1–W4 rollups byte-identical. **R3:** the corpus is built and locked on x86-64 Linux and regenerated byte-identically on Apple Silicon macOS.

Appendix C — Findings F1–F13

#	workload	finding (headline)
F1	W1	Semantic retrieval clears the recency floor: vector 1.000 vs floor 0.596 (med) / 0.358 (hard).
F2	W1	Efficiency frontier: 0.801 recall at a 32-char budget (precision 0.864); ~17× cost to reach 1.000.
F3	W2	Drift "budget illusion": vector collapses to 0.281 at a tight budget (vs 0.801 no-drift); generous-budget recall is a flood with stale versions.
F4	W2	`SUPERSESSION_CHAIN` recovers tight-budget recall 0.281→0.867 (+0.585 @ 32), embedder held constant.
F5	W2	`BI_TEMPORAL` reclaims 375 historical (`as_of`) queries/seed (hard): N/A → ~1.0.

#	workload	finding (headline)
F6	W3	Distractor noise is a precision tax: recall saturates to 1.000 while precision craters to 0.050.
F7	W3	The embedder is the lever (+0.510 recall @ 32: BGE 0.685 vs stub 0.175); capabilities inert (+0.000).
F8	W4	Recall-vs-footprint: unbounded flat 1.000 at linear cost; bounded (K=64) flat cost, overall recall 0.659→0.348.
F9	W4	Bounded-K forgetting cliff at exactly d = K = 64; structural and embedder-invariant.
F10	W6	`soft_delete` claims `HARD_DELETE` yet leaks at 1.000 (claim ≠ behavior); audit and retrieve disagree.
F11	W6	`coarse_delete` over-deletes: leakage 0.000 but survivor recall 0.000.
F12	W5	`leaky_tenant` claims `MULTI_TENANT` yet leaks at 1.000 (claim ≠ behavior); leakage orthogonal to recall (1.000).
F13	W5	`over_isolating` over-restricts: leakage 0.000 but in-tenant recall 0.000.

Appendix D — Reproduction

Environment: Python 3.12; reference embedder `BAAI/bge-small-en-v1.5` (pinned), exact NumPy cosine; retrieval budget enforced in characters.

Regenerate and verify the corpus — must report `corpus_hash 8ea8edc8…` and the six rollups of Appendix B:

```
python scripts/generate_corpus.py
```

Per-workload experiments (real BGE for the headline numbers):

```
python scripts/run_w1.py   # F1, F2        python scripts/run_w4.py   # F8, F9
python scripts/run_w2.py   # F3, F4, F5    python scripts/run_w5.py   # F12, F13
python scripts/run_w3.py   # F6, F7        python scripts/run_w6.py   # F10, F11
```

R3: regenerating on a second machine of different architecture must reproduce all six rollups and the corpus hash byte-for-byte.

References

Lucas Bourtoule, Varun Chandrasekaran, Christopher A. Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine unlearning. In *IEEE Symposium on Security and Privacy (S&P)*, 2021. arXiv:1912.03817.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025. ECAI 2025.

European Parliament and Council. Regulation (eu) 2016/679 (general data protection regulation), article 17: Right to erasure, 2016.

Cheng-Ping Hsieh et al. RULER: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024. CONFIRM.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2024. CONFIRM arXiv:2402.17753.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.

Preston Rasmussen et al. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

Richard T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 1999.

Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*, 2023.

Di Wu et al. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.

Wujiang Xu et al. A-Mem: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.